

Incoresoft VMS Plugin SDK

Developer Guide & Reference

Target release line: VMS 25.1 (Javalin 5, Java 21, Vue 3.4, Module Federation)

Generated: 2026-10-05

Contents

VMS Plugin SDK

- Contents
- Templates
- Reading order
- Deliverables
- License

00 — VMS Plugin SDK at a glance

- What a VMS plugin is
- What a plugin can do
- What a partner needs
- How it goes from idea to customers
- Rules of the game
- Where to read next

01 — Architecture

1. The host in one paragraph
2. What a plugin is
3. Lifecycle
4. How the web client loads a plugin
5. Shared singletons

02 — Getting started and the development loop

1. Prerequisites
2. The Sample Plugin
3. Thirty-minute walkthrough
4. The development loop
5. Mapping the sample to your product
6. Testing
7. Troubleshooting

03 — Backend

1. Project
2. Extension discovery
3. Map of the sample
4. `VmsExtension` — the contract
5. Wiring with Guice
6. Host services
7. Database schema
8. HTTP API
9. WebSocket and SSE
10. Serving the frontend bundle
11. Permissions
12. Alarm rules, alarms, notifications
13. Retention
14. Cameras and devices
15. Maps and plans
16. Logging, threads, configuration

04 — Frontend

1. Toolchain and packages
2. `webpack.config.js`
3. `public/component.json`
4. `public/settings.json`
5. Project layout and conventions
6. Talking to the backend
7. Host modules you can import
8. Permissions on the client
9. Locales
10. Slots

05 — Packaging, versioning, release

1. The artifact
2. Getting the frontend into the jar
3. Versioning
4. CI outline
5. Host HTTP API for plugin management

VMS Plugin SDK

Documentation and templates for building **third-party plugins for Incoresoft VMS**, release line **25.1**. Everything here was checked against the VMS host and the sample plugin that the templates form; where a statement is a recommendation rather than observed behaviour, the text says so.

Contents

#	Document	What you get
00	Overview	Two pages for decision makers: capabilities, requirements, process, rules
01	Architecture	What a plugin is, plugin types, identifiers, lifecycle, how the web client loads a plugin
02	Getting started	Prerequisites, the sample plugin in thirty minutes, the development loop, troubleshooting
03	Backend	<code>pom.xml</code> and manifest, <code>VmsExtension</code> , Guice, database, REST, WebSocket, host services, permissions, rules and alarms, retention, cameras, maps
04	Frontend	Module Federation setup, <code>component.json</code> , <code>settings.json</code> , layouts and routes, API calls, locales, every UI slot with its props and events
05	Packaging and release	Jar layout, versioning, CI outline, the host HTTP API for plugin management

Templates

Path	Purpose
<code>templates/backend-plugin-template/</code>	The sample plugin's backend for VMS 25.1: categories (with a VMS camera) managed in Admin Center, items created by operators, permissions, a rule type with alarms and notifications, a live WebSocket feed, retention, a paged and filterable API that also serves the Search page. Ships a build of the frontend template in <code>src/main/resources/public/</code> , so <code>mvn package</code> gives a complete plugin. Published as its own repository: vms-plugin-backend-template
<code>templates/frontend-plugin-minimal/</code>	The sample plugin's frontend: an Admin Center section, a client tab with pages and archive video, header "+" item, admin sidebar item, icon, rule-wizard blocks, alarm, notification, Search and layout-cell slots, locales <code>en / uk / es</code> . Installs from the public npm registry (<code>@incoresoft/incoresoft-ui</code> , <code>-icons</code> , <code>-toasts</code> ^25.1.0). Published as its own repository: vms-plugin-frontend-template
<code>tools/build-pdf.mjs</code>	Builds the PDFs (Developer Guide, Overview EN/UK) from the markdown: <code>node tools/build-pdf.mjs</code> (needs a <code>node_modules</code> with <code>micromark</code> and <code>micromark-extension-gfm</code> , see <code>MICROMARK_NODE_MODULES</code> in the script)

Reading order

New to VMS: 01 → 02, then 03 and 04 while coding, 05 when shipping.

Deliverables

File	Audience	Contents
VMS-Plugin-SDK-Overview.pdf	technical decision makers	docs/00-overview
VMS-Plugin-SDK-Developer-Guide.pdf and this repository	partner developers	README, docs/00-05, templates

The documentation mentions Incoresoft's own plugins (POS terminals, access control) as the origin of a pattern. Their sources are not part of this SDK; example plugin sources can be shared with partners on request.

License

Documentation: CC BY 4.0. Templates and tools: MIT. The Java libraries (`middleware`, `device-driver-sdk`) are published on Maven Central under the Apache License 2.0. The npm packages (`@incoresoft/incoresoft-ui`, `-icons`, `-toasts`) are distributed under the Incoresoft VMS Plugin SDK License shipped with them. See `LICENSE`.

Support: support@incoresoft.com

00 — VMS Plugin SDK at a glance

For technical decision makers evaluating a VMS integration. Two pages. The developer documentation starts in 01.

What a VMS plugin is

A plugin extends Incoresoft VMS from the inside. It is **one jar** containing a Java 21 backend extension and a Vue 3 frontend bundle. The VMS core loads it at runtime — no restart, no fork of the product — and users manage it in Admin Center or install it from the Incoresoft Marketplace.

What a plugin can do

Area	Capability
Own business logic	REST and WebSocket endpoints on the VMS server, protected by VMS authentication; own database tables in the VMS database; background processing; retention of its data through the standard storage cleaner
Own UI	A tab in the client application (any number of pages), a section in Admin Center, Video-wall cells, entries in the "+" menu and the admin sidebar, all built with the Incoresoft UI kit so they look native
Security model	Its own permissions, shown and granted in Admin Center → Roles like built-in ones, enforced by the server and respected by the UI
Events & Rules	New rule types for the alarm engine: users configure "when X happens, notify / record / trigger output", the plugin raises alarms and notifications with its own payload and its own alarm details UI
Maps & plans	Own object types placed on maps and floor plans (doors, terminals, sensors) with live status and event pop-ups
Search	Own result type in the global Search page with filters, cards and details
Platform data	Read access to cameras, streams, recording servers, users and roles; snapshots and video around an event

A plugin with your own video analytics is an ordinary plugin: your analytics server posts its detections to the plugin, the plugin stores them with the camera they came from, and VMS adds alarms with video, notifications and search. No Incoresoft analytics server is involved.

What a partner needs

Skill s	Java 21, Maven, Guice, Javalin, SQL (jOOQ/Liquibase); Vue 3, TypeScript, Webpack Module Federation
Tooling	JDK 21, Maven 3.9, Node.js 18+, a VMS 25.1 installation for development (evaluation licence from Incoresoft)
Libraries	The public Java host API (<code>com.incoresoft.commons:middleware</code> , plus <code>com.incoresoft:device-driver-sdk</code> for plugins that resolve cameras/devices) on Maven Central, and the UI packages <code>@incoresoft/incoresoft-ui</code> , <code>-icons</code> and <code>-toasts</code> on the public npm registry; nothing to configure on either side
SDK package	This documentation, a complete sample plugin as two templates (backend and frontend)

How it goes from idea to customers

1. **Kick-off** — partner receives credentials, the SDK package and a development VMS.
2. **Scaffold** — copy the two templates, rename, run them against the development VMS (first visible result in under an hour).
3. **Develop** — build the jar and upload it to the development VMS in Admin Center; the new version replaces the old one without restarting VMS and keeps the plugin's data and rules. The host finds the plugin's UI automatically.
4. **Package** — one `mvn package` produces the jar with the frontend inside.
5. **Deliver** — hand the jar to your customers: they install and update it in Admin Center (Settings → Marketplace → Extra Settings → Install Plugin from Disk) or over the API. A listing in the Incoresoft Marketplace, with one-click install and updates, is arranged with Incoresoft.

Rules of the game

- **One plugin version series per VMS release line.** VMS 25.1 accepts plugins versioned `25.1.x`; a new VMS line (e.g. 26.1) needs a rebuilt, re-versioned plugin. The SDK libraries follow the same lines and their API is kept binary compatible within a line.
- **Plugins run inside the VMS process with full trust.** That is what makes them powerful; it is also why partners are expected to follow the SDK's route, permission and resource conventions.
- **Identifiers are yours to choose once.** Plugin id, module name and permission ids are part of the customer's data after the first installation and must stay stable.
- **Languages.** The UI framework supports English, Ukrainian and Spanish; plugins ship their own translations.

Where to read next

`01 Architecture` for the technical picture, `02 Getting started` for the hands-on walkthrough, `04 Frontend` §10 for the complete list of UI extension points.

01 — Architecture

1. The host in one paragraph

Incoresoft VMS is one Java 21 process. It serves HTTP, WebSocket and SSE with **Javalin 5** on a single shared instance, keeps its data in **PostgreSQL** through **jOOQ** and **Liquibase**, and loads plugins at runtime with **PF4J**: every plugin is a jar in the `plugins/` directory with its own class loader. The web client is a **Vue 3** application assembled from **Webpack Module Federation** remotes; a plugin's frontend is one more remote, served from the plugin jar.

A plugin compiles against one small public library, `com.incoresoft.commons:middleware` (interfaces only: `VmsExtension`, `VmsExtensionContext`, `RuleManager`, `Permission`, `User`, ...). Plugins that resolve cameras and devices also use `com.incoresoft:device-driver-sdk` (`DeviceManager`, `Device`, `DeviceItemSettings`). Both are `provided` dependencies: the host already has them. Everything else a plugin needs (Guice bindings of the host services, serving the frontend bundle, registering permissions, parsing requests) is a few small classes that live in the plugin itself; the backend template contains all of them.

2. What a plugin is

One jar:

```
vms-sample-plugin-25.1.0.jar
├─ META-INF/MANIFEST.MF          ← PF4J descriptor: Plugin-Id, Plugin-Version, Plugin-Name, ...
├─ META-INF/extensions.idx       ← generated at compile time: the class name of your
VmsExtension                     VmsExtension
├─ com/example/vms/sample/**     ← your classes (+ third-party libraries the host does not
have)                             have)
├─ sql/db-changelog.xml          ← Liquibase changelog of your tables
├─ icon.svg                      ← Plugin-Icon-Resource-Path
├─ public/                      ← the frontend bundle
│   └─ app.js                   ← Module Federation container
│   └─ <chunks>.js
│   └─ component.json           ← which exposed components go into which UI slots
└─ settings.json                ← title, permissions, flags
```

The backend half is a subclass of `com.incoresoft.middleware.vms.plugin.VmsExtension`. The frontend half is a Module Federation remote whose container name equals the value the backend returns from `getPluginName()`. The host joins them through `GET /api/v1/plugins/extensions`, which the web client calls at start to learn which remotes to load and which components to mount where.

2.1 Plugin types

Type (type in the extensions API)	How the host decides	Who builds it
<code>service_plugin</code>	<code>getModuleClass()</code> returns <code>null</code>	Partners. Own REST API, own tables, own UI, own rule types. Incoresoft's POS-terminals and access-control plugins are of this kind.
<code>analytics_plugin</code>	<code>getModuleClass()</code> returns an analytics module class	Incoresoft only: plugins paired with Incoresoft's VEZHA analytics server.
<code>driver</code>	a <code>DeviceDriver</code> extension in <code>drivers/</code>	Device drivers; separate SDK, not covered here.

A plugin with **your own video analytics** is an ordinary service plugin: your analytics server posts its detections to your plugin's API, the plugin stores them with the camera they came from, and VMS does the rest — alarms with video, notifications, search, layout cells. Nothing in VMS requires the VEZHA server for that.

2.2 Identifiers

Identifier	Where it lives	Used for	Rules
Plugin-Id	<code>MANIFEST.MF</code> → <code>Plugin-Id</code>	Install, update, start, stop and uninstall API; the marketplace product id; the plugin's enabled state in the database; the icon URL <code>/resources/<Plugin-Id>/icon</code>	Unique across the marketplace. Convention: <code>vms-<name>-plugin</code> .
Plugin name (<code>module_name</code>)	<code>VmsExtension.getPluginName()</code> ; webpack name; <code>nameset.ts</code> → <code>PLUGIN_ID</code>	Module Federation container, bundle URL <code>/resources/<name>/app.js</code> , route prefixes <code>/<name>/...</code> and <code>/admin/<name>/...</code> , the key of every UI slot, <code>RuleType.getPluginName()</code>	A valid JavaScript identifier (<code>[A-Za-z_][A-Za-z0-9_]*</code>), lower snake_case by convention (<code>sample_plugin</code>).
Plugin-Version	manifest	Compatibility check on upload, marketplace updates	Strict SemVer and the same major.minor as the VMS release line: <code>25.1.<patch></code> for a 25.1 core, otherwise the upload answers <code>ERROR_PLUGIN_NOT_COMPATIBLE</code> .
Rule type id	<code>RuleType.getTypeId()</code>	The <code>type</code> of your alarms and notifications on the wire; the frontend picks your alarm components by it	Convention: equal to the plugin name.
Permission id	<code>Permission.getId()</code>	Roles UI, route roles, <code>settings.json</code> , <code>hasPluginPermission()</code>	Unique across VMS and all plugins, PascalCase with your plugin in it: <code>ManageSampleItems</code> .

The plugin id, the plugin name and the permission ids become part of the customer's data after the first installation. Choose them once.

3. Lifecycle

install / start	1. PF4J loads the jar and validates the manifest. 2. PF4J starts the plugin; the host checks that the jar contains a <code>VmsExtension</code> ("Plugin X misses expected extensions" otherwise). 3. The host remembers the plugin as enabled (database). 4. <code>ext.init(VmsExtensionContext)</code> ← YOUR CODE Every HTTP and WebSocket route added during <code>init()</code> is recorded. 5. The icon endpoint is registered; "plugin started" goes out over SSE,
	and open web clients offer to reload.
stop / update / uninstall	1. PF4J stops the plugin. 2. The host removes every recorded route. 3. <code>ext.terminate()</code> ← YOUR CODE 4. Uninstall only: unload, delete the jar.

What this means for you:

- **Routes are removed for you; nothing else is.** Unregister in `terminate()` what you registered in `init()`: permissions, the rule type, map emitters, listeners; stop your threads and close your sockets.
- If `init()` throws, the host stops the plugin again, which calls `terminate()` on the same instance; write `terminate()` so that it survives a half-finished `init()`. An uploaded jar whose start fails is deleted again.
- Plugins are hot-pluggable: install, update, start, stop and uninstall happen while VMS runs. An update replaces the jar in place and keeps the plugin's tables, settings and alarm rules. Open web clients need a page reload to pick up a new frontend bundle; the host asks them to.
- A plugin runs **inside the VMS process with its privileges**: a fault in the plugin is a fault in VMS, so guard every call to the host and follow the SDK's conventions.

4. How the web client loads a plugin

1. The client shell calls `GET /api/v1/plugins/extensions` and receives, for every installed plugin, its bundle URL (`/resources/<name>/app.js`), its `settings.json` values and the entries of its `component.json`.
2. It loads its own modules, then all plugin bundles in parallel. A plugin whose script or any declared component fails to load is marked as failed, all its components are dropped and the user sees a message naming it.
3. Entries of type `view` become routes: `/<name>/...` for the client, `/admin/<name>/...` for Admin Center. The child routes come from the modules `./clientRoutes` and `./adminRoutes` your bundle exposes.
4. Every other entry is put into a map keyed by plugin name and provided to the whole application; the host modules render `map[<name>]` in the matching slot. Chapter 04 lists the slots.
5. Translations: a module the host loads at start (the template: the layouts, through `import '@/locales'`) registers the plugin's messages. The host applies them once, after all plugins are loaded, and its own keys win on conflict — prefix yours.

5. Shared singletons

Vue, vue-router, vue-i18n, pinia and every `@incoresoft/*` package are Module Federation singletons **provided by the host** (`import: false` in the plugin's `shared` config). The plugin never bundles its own copy, so:

- build and type-check against the versions the host runs (the template pins them: Vue 3.4.15, vue-router 4.5.1, vue-i18n 9.9.1, pinia 2.1.7, @incoresoft/* ^25.1.0);
- the host's router, i18n and Pinia are yours to use (useRouter(), useI18n(), host stores);
- Pinia store ids, modal names and route names are global: prefix them with the plugin name.

02 — Getting started and the development loop

1. Prerequisites

Tool	Version	Why
JDK	21	The host and its libraries are compiled for 21
Maven	3.9+	Builds the plugin jar. Every dependency, <code>middleware 25.1.26</code> and <code>device-driver-sdk 25.1.27</code> included, comes from Maven Central; no repository to add, no credentials
Node.js	18+	Frontend build. The UI packages <code>@incoresoft/incoresoft-ui</code> , <code>-icons</code> and <code>-toasts</code> come from the public npm registry; nothing to configure
A VMS installation	25.1, build 25.1.1063 or newer	Your test system, including Admin Center access as the root user. You install and replace the plugin through Admin Center; no VMS source code is needed. Older 25.1 builds make you delete the plugin's alarm rules before every update

2. The Sample Plugin

The templates form one small but complete service plugin. Its data are **categories** and **items**; read them as "your devices or zones" and "events from your system" (§5).

- **Admin Center** is where the plugin is set up: a section with the categories table. A category has a name and, optionally, a VMS camera.
- **The client** is where operators work: a tab with the items of all categories, shown in pages, filtered by category, updated live for everyone. An item remembers its category's camera, so a click shows a frame and the archive video of the moment it was created.
- **Rules.** Operators build alarm rules on items (Admin Center → Alarm Rules): "Any item" in chosen categories, or "Item contains a word". A matching item raises an alarm, and a notification if the rule says so. When the category has a camera, the alarm shows its video.
- **Search.** The Search page gets a block for the plugin: items by period, category, text and camera, with a frame per result and the archive video in the details.
- **Layout cell.** A cell on a layout shows the newest items of one category, updated live.

3. Thirty-minute walkthrough

Step 1 — build the backend

```
cp -r vms-plugin-sdk/templates/backend-plugin-template vms-sample-plugin
cd vms-sample-plugin
mvn clean package                # → build/vms-sample-plugin-25.1.0.jar
```

The jar already contains a build of the frontend template under `public/`, so this is the whole plugin. What is in it (chapter 03 explains each piece):

- `PluginExtension` extends `VmsExtension` — the lifecycle; `PLUGIN_NAME = "sample_plugin"`.
- A Liquibase changelog creating `sample_categories` and `sample_items`.
- Three permissions (`ViewSampleItems`, `ManageSampleItems`, `ManageSampleCategories`) that appear in Admin Center → Roles.

- REST API `/api/v2/sample_plugin/categories` and `/api/v2/sample_plugin/items`, guarded by those permissions.
- `FrontendResources`, serving the frontend bundle at `/resources/sample_plugin/*`.
- A rule type with two triggers, scoped to categories, with alarms and notifications.
- A live WebSocket feed at `/api/v2/ws/sample_plugin/items`.
- A retention task that deletes old items on the schedule configured for plugin data.

Step 2 — install

Admin Center → **Settings** → **Marketplace** → **Extra Settings** → **Install Plugin from Disk**, pick the jar. VMS starts the plugin at once; no restart.

The same over the API, with an API token of a **root** user (Admin Center → Settings → API Token; plugin management is root-only):

```
TOKEN=<api token>
curl -k -X POST https://<vms-host>:2443/api/v1/plugins/vezha \
  -H "Authorization: Bearer $TOKEN" -F file=@build/vms-sample-plugin-25.1.0.jar
```

Verify:

```
curl -k https://<vms-host>:2443/api/v1/plugins/extensions | jq '.[0] |
select(.module_name=="sample_plugin")'
curl -k -H "Authorization: Bearer $TOKEN" https://<vms-host>:2443/api/v2/sample_plugin/items
```

The first call shows your extension with `type: "service_plugin"` and `plugin_id: "vms-sample-plugin"`. The second answers `{"data":[], "total":0, "pages":0}` (no items yet), or `403` until a role of the token's user holds `ViewSampleItems`; grant it in Admin Center → Roles, where the permission appears under the plugin name. If the upload answered `400 ERROR_PLUGIN_NOT_COMPATIBLE`, the `Plugin-Version` does not share major.minor with the core (the template uses `25.1.0`).

Step 3 — try it

Reload the web client: a page that was open during the install does not see a new plugin by itself.

- **Admin Center:** the plugin is in the sidebar under **PLUGINS**. Create a category; pick a camera if you want video on its alarms.
- **Client:** click **+** in the header → "Sample Plugin" → the items page opens. Create an item: it appears at once in every open items page and layout cell. The play button in its row shows the frame and the archive video from the camera.
- **Admin Center** → **Alarm Rules:** create a rule of type "Sample Plugin", pick a trigger and, for "Any item", the categories. The next matching item raises an alarm; add the notification action to get a notification too.
- **Search:** open the Search page, choose a period, and the "Sample Plugin" block in the sidebar lists the items with a frame each.
- **Layout:** the layout sidebar has a "Sample Plugin" entry in the **Plugins** group. It asks for a category and adds a cell with that category's newest items.

Step 4 — change the frontend and ship it

```
cp -r vms-plugin-sdk/templates/frontend-plugin-minimal frontend-sample
cd frontend-sample && npm install
npm run typecheck                # vue-tsc, .vue files included
npm run build                     # production bundle in dist/
rm -rf ../vms-sample-plugin/src/main/resources/public
mkdir -p ../vms-sample-plugin/src/main/resources/public
cp -r dist/* ../vms-sample-plugin/src/main/resources/public/
cd ../vms-sample-plugin && mvn clean package
```

Upload the new jar as in Step 2. VMS sees that the plugin is installed and asks whether to reinstall it; confirm. The new jar replaces the old one while VMS runs: the plugin's tables, its data and its alarm rules stay. Reload the page to load the new bundle.

4. The development loop

Every change, frontend or backend, reaches VMS as a new jar. The web client shipped with VMS loads a plugin's frontend only from the jar (`/resources/<name>/app.js`); the `dev_path` in `settings.json` is used by Incoresoft's internal development build of the client and has no effect on an installed VMS.

1. Frontend: `npm run typecheck`, `npm run build` → `dist/`. Always a production build: a development build points its chunk URLs at a local dev server and does not load inside VMS.
2. Replace the content of the backend's `src/main/resources/public/` with `dist/*`.
3. Backend: `mvn clean package` → `build/<artifactId>-<version>.jar`.
4. Upload the jar in Admin Center and confirm the reinstall. The host stops the plugin (`terminate()`), replaces the jar, starts the new one (`init()`), and keeps the old jar as `.bak` next to it. If the new jar fails to start, the old one is put back.
5. Reload the browser page.

A backend-only change skips steps 1–2. Over the API, the update of an installed plugin is `PATCH /api/v1/plugins/vezha/<Plugin-Id>/update_from_file` with the jar as multipart file (chapter 05).

Action	How
First install	Admin Center → Settings → Marketplace → Extra Settings → Install Plugin from Disk, or <code>POST /api/v1/plugins/vezha</code>
Replace with a new build	Upload again and confirm the reinstall prompt, or <code>PATCH ../vezha/<Plugin-Id>/update_from_file</code>
Stop / start without uninstalling	Admin Center → Settings → Marketplace, or <code>PATCH ../vezha/<Plugin-Id>/stop</code> , <code>../start</code> . The state survives a VMS restart
Uninstall	Admin Center, or <code>DELETE ../vezha/<Plugin-Id></code> . Refused while alarm rules of the plugin's rule type exist (<code>ERROR_CANNOT_DELETE_PLUGIN_WITH_EXISTING_RULES</code>); delete them first
Watch what happened	<code>log/application.log</code> in the VMS working directory: <code>Plugin <id> is loaded.</code> , <code>Plugin <id> misses expected extensions</code> , <code>Couldn't start plugin with the stack trace</code>

Never overwrite a jar in the `plugins/` directory while VMS runs. The JVM keeps the file open and the running plugin breaks (`ZipException`), with HTTP 500 on `GET /api/v1/plugins/extensions` until VMS is restarted. Copying a jar there is fine while VMS is stopped; it is loaded at the next start, without the version check the upload performs.

5. Mapping the sample to your product

The sample is a map of the extension points, not a product. When you copy it:

Sample	Your plugin
Category (set up by the administrator, bound to a camera)	A door, a cash desk, a sensor, a zone watched by your analytics — any object your system reports about
Item (created by operators or by your system through the API)	An event from your system: an access, a receipt, a detection
"Any item" / "Item contains a word"	Your triggers: "any event", "event of type X", "amount over N"
Camera of the category → video on alarms, in the list and in Search	Keep it: it is what makes your data useful in a VMS

Rename the identifiers (03 §1.3 and the frontend template's README have checklists), replace the fields, keep the structure.

6. Testing

- Backend: JUnit 5 and Mockito work as in any Java project; mock the `VmsExtensionContext` interfaces. There is no host test harness.
- Frontend: the templates ship no test runner; add the one you use. For end-to-end tests the template puts `data-el-name` attributes (`${TESTING_ID}-...`) on the elements a test needs.
- API: any HTTP client with `Authorization: Bearer <api token>`. The host serves no Swagger for plugins; add your own docs route if you want one.

7. Troubleshooting

Symptom	Cause / fix
Plugin X misses expected extensions in the log	No <code>META-INF/extensions.idx</code> , or it lists no <code>VmsExtension</code> subclass. Keep <code>pf4j</code> on the compile classpath and annotation processing on; check the jar
Upload answers 400 <code>ERROR_PLUGIN_IS_INVALID</code>	<code>Plugin-Id</code> missing, or <code>Plugin-Version</code> is not strict SemVer
Upload answers 400 <code>ERROR_PLUGIN_NOT_COMPATIBLE</code>	<code>Plugin-Version</code> major.minor differs from the core's. Version your plugin <code>25.1.N</code>
Upload answers 400 <code>ERROR_PLUGIN_IS_ALREADY_INSTALLED</code>	A plugin with this <code>Plugin-Id</code> is installed. Use the update (<code>update_from_file</code>); Admin Center offers it as "reinstall"
Upload answers 403	The user is not root. Plugin management needs the root user
Upload answers 500 <code>ERROR_PLUGIN_LOADING_FAILED</code> and the jar is gone	<code>init()</code> threw, or the plugin could not be loaded; the log has the cause
Plugin listed but never starts, log says <code>requires a minimum system version</code>	<code>Plugin-Requires</code> does not match the core; PF4J keeps the plugin disabled. Drop the attribute
Plugin starts, no "+" entry, no admin section	Page not reloaded after the upload; or the user lacks the permission in <code>client_permissions.plugin_page</code> ; or a component failed to load (browser console: <code>[Extension Error]</code>)
<code>[Extension Error]</code> Failed to load component X	<code>component</code> in <code>component.json</code> is not an <code>exposes</code> key of the bundle, or the module has no default export

Symptom	Cause / fix
Plugin UI requests chunks from <code>localhost</code>	The bundle was built in development mode. <code>npm run build</code>
Frontend change not visible	<code>dist/</code> not copied into <code>src/main/resources/public/</code> before <code>mvn package</code> , jar not uploaded again, or page not reloaded
Request answers 400 with a key like <code>SAMPLE_CATEGORY_NOT_FOUND</code> for a filter you did not set	A query parameter went out as <code>name=undefined</code> : the host writes every key of <code>params</code> into the URL, unset ones included. Add only parameters that have a value
Alarm created but no message shown	<code>alarms_message_column</code> / <code>alarms_details_message</code> not registered for your type, or the alarm JSON does not match your component
Rule "What" step empty for your type	Missing <code>admin_events_and_rules_options_block</code>
Permission shows its raw key in Roles	Add the <code>PERMISSION_*</code> translation to your locales
Texts show raw keys	Locales registered after start (register them from a module the host loads at start, as the template's layouts do), or an unprefixed key overridden by a host key
Tab state lost on every tab switch	The client layout's component <code>name</code> is not <code>ClientLayout<PascalCase(name)></code>
Alarms and notifications stop when you call the host	A host lookup threw inside your listener. Guard calls to the host (<code>DeviceManager</code> etc.) with try/catch and log; an alarm without extra details beats no alarm

03 — Backend

Reference implementation: `templates/backend-plugin-template`. The patterns are the ones Incoresoft's own plugins use. This chapter describes the VMS **25.1** host (Java 21, Javalin 5.6.3).

1. Project

1.1 `pom.xml`

Everything the host already has is `provided`; the template pins these versions:

```
<dependency> <groupId>com.incoresoft.commons</groupId> <artifactId>middleware</artifactId>
<version>25.1.26</version> <scope>provided</scope> </dependency>
<dependency> <groupId>com.incoresoft</groupId> <artifactId>device-driver-
sdk</artifactId> <version>25.1.27</version> <scope>provided</scope> <!--
cameras, devices -->
<dependency> <groupId>io.javalin</groupId> <artifactId>javalin</artifactId>
<version>5.6.3</version> <scope>provided</scope> </dependency>
<dependency> <groupId>com.google.inject</groupId> <artifactId>guice</artifactId>
<version>7.0.0</version> <scope>provided</scope> </dependency>
<dependency> <groupId>org.jooq</groupId> <artifactId>jooq</artifactId>
<version>3.18.7</version> <scope>provided</scope> </dependency>
<dependency> <groupId>com.fasterxml.jackson.core</groupId> <artifactId>jackson-
databind</artifactId> <version>2.14.2</version> <scope>provided</scope> </dependency>
<dependency> <groupId>org.slf4j</groupId> <artifactId>slf4j-api</artifactId>
<version>2.0.17</version> <scope>provided</scope> </dependency>
<dependency> <groupId>javax.validation</groupId> <artifactId>validation-api</artifactId>
<version>2.0.1.Final</version> <scope>provided</scope> </dependency>
<dependency> <groupId>org.pf4j</groupId> <artifactId>pf4j</artifactId>
<version>3.8.0</version> <scope>provided</scope> </dependency>
```

Any 25.1.x `middleware` and `device-driver-sdk` work together; never mix them with libraries of another release line (26.1 moves to Javalin 7).

What runs is the host's copy of each library: on 25.1 that is Jackson **2.14.2**, Guice 7.0.0, jOOQ 3.18.7, Liquibase 4.28, Javalin 5.6.3. Call only API that exists in those versions. Bundle **only** libraries the host does not have; a second copy of a host library inside your jar is loaded by your class loader and ends in `ClassCastException`s at the boundary.

jOOQ: the host runs the Professional edition, whose packages are identical to the open-source `org.jooq:jooq`. Compile against the open-source artifact; at runtime the host's classes are used. The template uses jOOQ's dynamic API (`table(name(...))`, `field(name(...))`) and needs no code generation.

1.2 Manifest

The assembly plugin (`jar-with-dependencies`, output directory `build/`) writes the PF4J descriptor:

Attribute	Required	Meaning
Plugin-Id	yes	Unique id, equals the marketplace product id. The template uses the Maven <code>artifactId</code> (<code>vms-sample-plugin</code>).
Plugin-Version	yes	Strict SemVer; major.minor must equal the core's release line (25.1.x), otherwise the upload answers 400 <code>ERROR_PLUGIN_NOT_COMPATIBLE</code> . A missing <code>Plugin-Id</code> or a non-SemVer version answers 400 <code>ERROR_PLUGIN_IS_INVALID</code> .

Attribute	Required	Meaning
Plugin-Name	yes	Display name in Admin Center → Settings → Marketplace.
Plugin-Description	recommended	Free text or a locale key of your frontend (the template: <code>SAMPLE_PLUGIN_DESCRIPTION</code>).
Plugin-Provider	recommended	Vendor name.
Plugin-Icon-Resource-Path	recommended	Classpath path of an SVG or PNG, served by the host at <code>GET /resources/<Plugin-Id>/icon</code> .
Plugin-Requires	leave out	A PF4J version constraint. When it does not match the core, PF4J keeps the plugin disabled with only a warning in the log. The major.minor rule already pins the release line, so the template does not set it.

`GET /api/v1/plugins/installed` reports `id`, `name`, `description` and `version` of a plugin; other manifest attributes are not exposed by any host API.

1.3 Rename checklist

1. `pom.xml`: `groupId`, `artifactId` (= `Plugin-Id`), `plugin.name`, `plugin.provider`.
2. `PluginExtension.PLUGIN_NAME` and the Java package.
3. Table prefix and `logicalFilePath` in `db-changelog.xml`, table names in the repositories.
4. Permission ids and locale keys in `SamplePermissions` (translations live in the frontend).
5. Trigger wire names in `rules/Trigger` and `getTypeLocaleKey()` in `SampleRuleType`, together with the frontend's `models/rules.ts` and locales.

2. Extension discovery

The host asks PF4J for every `VmsExtension` in your jar. PF4J finds extension classes through `META-INF/extensions.idx`, generated at **compile time** by PF4J's annotation processor, which `javac` picks up whenever `pf4j.jar` is on the compile classpath. `VmsExtension` carries an inherited `@Extension` annotation, so your subclass is indexed without any annotation of its own. Verify after `mvn package`:

```
unzip -p build/vms-sample-plugin-25.1.0.jar META-INF/extensions.idx
# # Generated by PF4J
# com.example.vms.sample.PluginExtension
```

Without the index (annotation processing off, or a build tool without the processor) the host logs `Plugin <id> misses expected extensions` and does not start the plugin.

3. Map of the sample

Package / file	What it shows
<code>PluginExtension</code>	Lifecycle: registering the rule-options deserializer, applying the schema, registering permissions, the rule type and the retention task; undoing it in <code>terminate()</code>
<code>PluginModule</code>	Guice bindings; which classes are eager singletons and why

Package / file	What it shows
host/HostServicesModule , host/CleaningCategories	Host services from <code>VmsExtensionContext</code> as Guice bindings; the host's retention categories
permissions/	<code>SamplePermissions</code> (<code>ViewSampleItems</code> , <code>ManageSampleItems</code> , <code>ManageSampleCategories</code>) and their registration with the host
db/ , resources/sql/db-changelog.xml	Liquibase schema in the shared database: <code>sample_categories</code> (name, camera) and <code>sample_items</code> (text, category, the camera copied from the category); a second changeSet adding columns; <code>TIMESTAMP(3)</code> columns; jOOQ without code generation; <code>ItemsQuery</code> for the list filters
dto/	Wire format: snake_case keys, epoch milliseconds; <code>PageDTO</code> for paged lists
http/CategoriesController	<code>/api/v2/sample_plugin/categories</code> : reading needs <code>ViewSampleItems</code> , writing <code>ManageSampleCategories</code> ; the camera is validated through <code>DeviceManager</code> ; the rule-source cache is dropped on every change
http/ItemsController	<code>/api/v2/sample_plugin/items</code> : paged list with the filters the client page and the Search page need (<code>category_id</code> / <code>category_ids</code> , <code>camera_ids</code> , <code>text</code> , <code>start_date</code> , <code>end_date</code> , <code>sort_order</code> , <code>limit</code> , <code>offset</code>), creation, deletion, domain events
http/Requests	Request parsing that answers 400/404 in the host's error format instead of the host's catch-all 500; list parameters as the host frontend sends them (<code>ids=[1,2]</code>)
http/FrontendResources	Serving the frontend bundle and the icon at <code>/resources/<plugin_name>/...</code>
core/ItemEventBus	Fan-out inside the plugin: the HTTP layer knows nothing about rules or WebSockets
rules/	A rule type whose sources are the categories (with the category's camera, so alarms show video), two triggers with polymorphic options, the alarm payload, the bridge that fires rules
ws/	A live WebSocket feed: permission check on connect and on every send, heartbeats, a scheduler stopped on terminate

4. `VmsExtension` — the contract

```
public abstract class VmsExtension extends SystemExtension<VmsExtensionContext> { }

// SystemExtension, middleware 25.1:
public abstract String getPluginName(); // module name – a
JavaScript identifier
public abstract Class<? extends AnalyticsModule> getModuleClass(); // return null:
service plugin
public abstract List<Table<?>> getManagedTables(); // your jOOQ tables
(included in backups)
public List<Table<?>> getBackupIgnoredTables() { return List.of(); } // tables to leave
out of backups
public List<String> getFrontComponents() { ... } // reads
public/component.json – do not override
public JSONObject getSettings() { ... } // reads
public/settings.json – do not override
// init(VmsExtensionContext) and terminate() from the lifecycle (§3)
```

Rules:

- Public no-arg constructor, non-abstract class.

- `init()` runs on the host's plugin-start thread: keep it fast, start background work on your own executor and stop it in `terminate()`.
- If `init()` throws, the host stops the plugin and calls `terminate()` on the same instance; keep the fields you need before registering anything and null-check them in `terminate()`.
- `getPluginName()` is called often; return a constant.
- `getManagedTables()`: list the tables your changelog creates, so system backups include them.
- Do not override `customAuditActions()`: the host never unregisters custom audit actions, so the plugin would fail to start again until VMS restarts.

What happens on stop, update or uninstall:

- Removed by the host: the HTTP and WebSocket routes registered during `init()`.
- Must be undone in `terminate()`: permissions, the rule type (which also removes the notification type), map emitters, listeners, your threads and executors, open WebSocket sessions.
- Not undone by anyone until VMS restarts: modules registered on the host's `ObjectMapper`, `Cleaner` tasks (there is no remove call), Javalin `exception(...)` handlers. Keep them idempotent and harmless when your plugin is gone.

5. Wiring with Guice

```
Injector injector = Guice.createInjector(new HostServicesModule(context, this))
                          .createChildInjector(new PluginModule(context));
```

`HostServicesModule` binds the host services from the context so you can `@Inject` them anywhere: `Javalin`, `DSLContext`, `DatabaseConnector`, `UsersManager`, `VmsPluginRolesManager`, `RuleManager`, `VmsNotificationManager`, `MapPlanEventManager`, `Cleaner`, `GlobalSettings`, `StreamsManager`, `Storage`, `DeviceManager` and `VmsExtension` (yourself). Any other context getter is one more line:
`bind(X.class).toInstance(context.getX())`.

Services and controllers are `asEagerSingleton()`: controllers register their routes in the constructor or in an `@Inject public void init(Javalin app)` method, so creating the injector wires the whole plugin. A listener that subscribes to the in-plugin event bus must be an eager singleton too, otherwise nothing constructs it.

6. Host services

What `VmsExtensionContext` gives a service plugin (types from `middleware` unless stated):

Getter	Type	Purpose
<code>getJavalin()</code>	<code>io.javalin.Javalin</code>	Register HTTP, WebSocket and SSE routes
<code>getDSLContext()</code>	<code>org.jooq.DSLContext</code>	SQL on the shared database
<code>getDatabaseConnector()</code>	<code>DatabaseConnector</code>	<code>checkStructure(changelog, classLoader)</code> , <code>getDataSource()</code> , <code>getDBType()</code>
<code>getUsersManager()</code>	<code>UsersManager</code>	<code>getUser(Context)</code> , <code>getUser(WsContext)</code> , <code>getUser(token)</code> , <code>getAll()</code>

Getter	Type	Purpose
<code>getRolesManager()</code>	<code>VmsPluginRolesManager</code>	<code>register(pluginName, PluginPermissions)</code> , <code>unregister(...)</code> , <code>getAll()</code> , <code>getPermissions()</code>
<code>getRuleManager()</code>	<code>RuleManager</code>	Alarm rules: <code>registerRuleType</code> , <code>unregisterRuleType</code> , <code>getRulesOfType</code> , <code>triggerRule(rule, alarm)</code>
<code>getNotificationsManager()</code>	<code>VmsNotificationManager</code>	<code>registerNotificationType</code> , <code>addNotification</code>
<code>getMapPlanEventManager()</code>	<code>MapPlanEventManager</code>	Objects and live events on maps and plans
<code>getDeviceManager()</code>	<code>com.incoresoft.vms.drivers.sdk.DeviceManager</code>	Cameras and devices: <code>getItem(id)</code> , <code>getDeviceByItemId(id)</code> , <code>getCameraByStreamUuid</code> , <code>isCameraAllowed(user, cameraId)</code>
<code>getStreamsManager()</code>	<code>StreamsManager</code>	Video streams: <code>get(uuid)</code> , <code>getAll(user)</code> , <code>createSnapshot(uuid)</code> , folders, <code>StreamListener</code>
<code>getStorage()</code>	<code>Storage</code>	<code>get(path)</code> , <code>save(path, File</code>
<code>getGlobalSettings()</code>	<code>GlobalSettings</code>	<code>getSettingsMap()</code> , <code>getValue(key)</code> , <code>addListener(OnSettingsChangeListener)</code>
<code>getCleaner()</code>	<code>Cleaner</code>	Retention tasks (§13)
<code>getObjectMapper()</code>	<code>Jackson ObjectMapper</code>	The host's mapper; register your (de)serializers on it

The context has more getters (message queue, analytics, VEZHA integration, audit). They serve Incoresoft's own plugins and are not part of the supported surface for partners; `getSettings()` and `getEventManagerConnector()` throw `UnsupportedOperationException` on 25.1.

Users and permissions:

```

interface User extends ClientId {
    int getId(); String getEmail(); String getFullname(); int getRoleId(); boolean isRoot();
    boolean hasPermission(Permission); boolean hasAllPermissions(Collection<? extends
Permission>);
    boolean hasAnyPermission(Collection<? extends Permission>);
    default ZoneId getTimezone(); default DayOfWeek getFirstDayOfWeek(); default boolean
hasAccessTo(ClientId);
}
interface UsersManager { // also get(Integer), getAll()
    User getUser(io.javalin.http.Context) throws UnauthorizedResponse; // throws without
a valid session or token
    User getUser(io.javalin.websocket.WsContext); // 25.1 host:
null when the session is gone
}
interface Permission extends io.javalin.security.RouteRole { String getId(); String
getLocaleKey(); Set<String> getRequired(); }

```

7. Database schema

```

boolean ok = context.getDatabaseConnector().checkStructure("sql/db-changelog.xml",
getClass().getClassLoader());
if (!ok) throw new IllegalStateException("Cannot check/update database structure of " +
PLUGIN_NAME);

```

- Liquibase runs against the **shared VMS database** on every plugin start. Prefix table names with your plugin (`sample_categories`, `sample_items`).
- Every plugin ships its changelog as `sql/db-changelog.xml`, and Liquibase identifies a changeSet by id + author + file path. Give the changelog a unique `logicalFilePath` (the template: `sample_plugin/db-changelog.xml`). Never edit an applied changeSet; add a new one, as the template's `sample-003-cameras` does.
- Keep `objectQuotingStrategy="QUOTE_ALL_OBJECTS"` and the `${now}` property pattern from the template: they keep the changelog portable across database types.
- `INT` for user ids (`User.getId()` is `int`) and camera ids, `TIMESTAMP(3)` for instants written as `Timestamp.from(Instant)`, the same convention as the core; the host's retention task compares the column in the server's time zone.

8. HTTP API

```

@Inject
public void init(Javalin app) {
    app.routes(() -> path(API_PATH, () -> { //
"/api/v2/sample_plugin/items"
        get(this::list, LOGGED_IN, API_CALL, VIEW_SAMPLE_ITEMS);
        post(this::create, LOGGED_IN, API_CALL, MANAGE_SAMPLE_ITEMS);
        path("{id}", () -> {
            get(this::getOne, LOGGED_IN, API_CALL, VIEW_SAMPLE_ITEMS);
            delete(this::remove, LOGGED_IN, API_CALL, MANAGE_SAMPLE_ITEMS);
        });
    }));
}

```

`get`, `post`, `put`, `patch`, `delete` and `path` are the static methods of `io.javalin.apibuilder.ApiBuilder`.

- **Path prefix** `/api/v2/<module_name>/...` `/api/v1/*` belongs to the core. Never register a wildcard outside your prefix.
- **Roles** (`com.incoresoft.middleware.http.AppRole`): `ANYONE` = no authentication. `LOGGED_IN` and `API_CALL` trigger the same check: a session cookie **or** an `Authorization: Bearer <api token>` header is accepted. Incoresoft's code lists both to say "for the UI and for integrations". Every `Permission` in the role list is required; a missing one answers 403. The root user passes every permission check.
- **Current user:** `userManager.getUser(ctx); user.hasPermission(p), user.getTimezone(), user.getClientId()`.
- **JSON:** `ctx.json(obj)` and `ctx.bodyAsClass(Dto.class)` use the host's Jackson mapper. It uses `SNAKE_CASE` names, ignores unknown properties and writes `Instant` as epoch milliseconds. The template annotates every field with `@JsonProperty` and keeps timestamps as `long` milliseconds in Java records. Register (de)serializers for your own classes on `context.get ObjectMapper()` in `init()`; they stay registered after your plugin stops, so register only your own types.
- **Query parameters:** the host frontend writes arrays as `ids=[1,2]` and writes **every** key it is given, unset ones as `name=undefined`. The template's `Requests.listParam` parses the bracket form (and plain `1,2`); on the frontend side the template adds only parameters that have a value.
- **Paged lists:** `?limit=&offset=` in, `{"data": [...], "total": n, "pages": n}` out, the shape the host uses for its own lists (template: `PageDTO`).
- **Errors:** the body of every error is the array `[{"type", "field", "args"}]`. `type` is a locale key the frontend translates, `field` names the input (`null` for a general error), `args` fills placeholders. With a `field` the host UI shows the message under that input; without one, a toast. Plugin keys (`SAMPLE_...`) are translated by your frontend locales, host keys by the host.

Thrown	Status	type
<code>com.incoresoft.middleware.http.exceptions.ValidationException(type, field, args...)</code>	400	your key, or a host key such as <code>FIELD_REQUIRED</code>
Javalin <code>NotFoundResponse</code>	404	<code>ERROR_NOT_FOUND</code>
Javalin <code>ForbiddenResponse</code>	403	<code>ERROR_FORBIDDEN</code>
Javalin <code>UnauthorizedResponse</code>	401	<code>ERROR_UNAUTHORIZED</code>
Javalin <code>BadRequestResponse</code>	400	<code>ERROR_BAD_REQUEST</code>
A body <code>ctx.bodyAsClass</code> cannot deserialize	498	<code>ERROR_DESERIALIZATION_FAILED</code>
Anything else	500	<code>ERROR_FAILED_TO_PROCESS_REQUEST</code>

- **Validation:** the template checks input by hand. `POST /items` with a blank `text` answers 400 `[{"type":"FIELD_REQUIRED","field":"text","args":[]}]`; a category with items cannot be deleted: 400 `[{"type":"SAMPLE_CATEGORY_IN_USE","field":null,"args":[]}]`. `http/Requests` turns a body it cannot read into 400 `ERROR_DESERIALIZATION_FAILED` and a non-numeric `{id}` into 404, so parse errors never reach the catch-all 500.
- **Exception handlers:** prefer throwing `ValidationException`. `app.exception(...)` handlers are global on the shared Javalin and stay registered after your plugin stops; map only your own exception classes.

9. WebSocket and SSE

```
app.ws("/api/v2/ws/sample_plugin/items", ws -> {
    ws.onConnect(ctx -> {
        User user = userManager.getUser(ctx); // null when the session is gone
        if (user == null || !user.hasPermission(VIEW_SAMPLE_ITEMS)) { ctx.closeSession(1008,
"Forbidden"); return; }
        ctx.enableAutomaticPings();
        sessions.put(ctx, user.getId());
    });
    ws.onClose(ctx -> sessions.remove(ctx));
    ws.onError(ctx -> sessions.remove(ctx));
}, LOGGED_IN, API_CALL);
```

The route roles authenticate the upgrade request but check no plugin permission; check it in `onConnect` and close with 1008 when it is missing. The template's `ws/ItemsWebSocketController` is the whole pattern: a session map filled after the check and emptied on close **and** error, `broadcast` = serialise once and send to every session, re-checking the user's permission on every send (permissions can be revoked while the socket is open), a heartbeat every 30 s (`{"type":"HEALTH_CHECK"}`), and `terminate()` that stops the scheduler and closes the sessions.

Frontend side: `${coreApi.config.WEBSOCKET_API}/api/v2/ws/<name>/...`; the host's `styleguide/services/WebSocketService` reconnects and parses JSON (template: `composables/useItemsLive.ts`).

10. Serving the frontend bundle

The web client loads a plugin from fixed URLs; the plugin serves them itself. The template's `http/FrontendResources` registers both routes in its constructor:

Route	Source
GET <code>/resources/<name>/*</code>	files under <code>public/</code> on the plugin class loader (<code>app.js</code> , <code>chunks</code> , <code>component.json</code> , <code>settings.json</code>); <code>Cache-Control: public, max-age=3600</code> for everything but <code>app.js</code>
GET <code>/resources/<name>/icon</code>	<code>icon.svg</code> at the jar root

Both routes are `ANYONE`: the browser fetches them before login.

11. Permissions

Permissions are `Permission` objects registered per plugin with `VmsPluginRolesManager.register(pluginName, PluginPermissions)`. The host shows them in Admin Center → Roles under the plugin's name, with the label from `getLocaleKey()` translated by your frontend, stores grants per role, and enforces them as route roles,

through `User.hasPermission` and on the frontend through `usePermissionStore().hasPluginPermission(id)` and `settings.json`.

```
public enum SamplePermissions implements Permission {
    VIEW_SAMPLE_ITEMS("ViewSampleItems", "PERMISSION_VIEW_SAMPLE_ITEMS"),
    MANAGE_SAMPLE_ITEMS("ManageSampleItems", "PERMISSION_MANAGE_SAMPLE_ITEMS",
VIEW_SAMPLE_ITEMS),
    MANAGE_SAMPLE_CATEGORIES("ManageSampleCategories", "PERMISSION_MANAGE_SAMPLE_CATEGORIES",
VIEW_SAMPLE_ITEMS);
    // getId(), getLocaleKey(), getRequired() – "Manage requires View": the Roles UI selects
them together
}
rolesManager.register(PLUGIN_NAME, PLUGIN_PERMISSIONS);    // init()
rolesManager.unregister(PLUGIN_NAME, PLUGIN_PERMISSIONS);  // terminate(), the same object
```

Permission ids are global across VMS and all plugins: keep your plugin's name in them.

12. Alarm rules, alarms, notifications

Operators create **rules** in Admin Center → Alarm Rules: a type with options ("What"), sources ("Where"), a schedule ("When") and actions (alarm, notification, recording, ...). A plugin adds rule **types**; the host runs the actions.

12.1 The rule type

One class implements `RuleType` and `VmsNotificationType` (template: `rules/SampleRuleType`), so one type id covers alarms and notifications, and registering it with `RuleManager` registers the notification type too.

Method	Contract
<code>getTypeId()</code>	The <code>type</code> of your alarms and notifications on the wire; the frontend picks your alarm components by it. Use the plugin name.
<code>getTypeLocaleKey()</code> , <code>getPluginName()</code>	Label in the rule wizard; the plugin whose "What" block is rendered.
<code>getRuleDataClass()</code>	Jackson target of <code>rule.options</code> . Usually polymorphic on a <code>trigger</code> field, with a deserializer registered on <code>context.getMapper()</code> in <code>init()</code> . The wizard sends <code>options</code> before the user picked a trigger: return <code>null</code> for a missing or unknown trigger instead of throwing.
<code>getRuleMessageClass()</code> , <code>getNotificationMessageClass()</code>	Jackson target of <code>alarm.message</code> ; any JSON your alarm components understand.
<code>getSources()</code>	The objects a rule can be scoped to in the "Where" step (the template: the categories). Each <code>RuleSource</code> has <code>getId()</code> (a stable UUID stored in rules and carried by alarms), <code>getName()</code> (shown as the alarm's source), <code>isAllowed(user)</code> (who may see alarms of this source) and <code>getCameraId()</code> . Called on hot paths (per alarm, per user): cache it.
<code>getCameraId()</code> on a <code>RuleSource</code>	The VMS camera behind the source. The host attaches it to every alarm of the source as <code>camera_id</code> , together with the camera's recording server, which is what gives the alarm its video.
<code>getDynamicSources(options)</code>	Source UUIDs a rule covers by itself, added to the ones picked in "Where". The template returns every category for "Item contains a word".
<code>requiresSources(options)</code>	<code>false</code> for triggers without object scope; the rule is then saved without sources.

12.2 Firing

```
for (Rule rule : ruleManager.getRulesOfType(ruleType)) {
    if (!matches(item, rule.getOptions())) continue;
    ruleManager.triggerRule(rule, new SampleAlarm(sourceId, serverId, message, recipients,
Instant.ofEpochMilli(item.createdAt())));
}
```

`triggerRule` ignores the call when the rule is disabled or outside its schedule, and **silently drops** an alarm whose `getSourceId()` is not among the rule's sources (the "Where" selection plus `getDynamicSources`). Then it creates the alarm, the notification if the rule has that action, and runs the other actions. `Alarm` is five getters: `getSourceId()`, `getServerId()` (the recording server UUID of the related camera, or `null`), `getMessage()`, `getAllowedUserIds()` (who may see it; keep it consistent with `RuleSource.isAllowed`) and `getTimestamp()` (when it happened).

Anything you call on the host while firing (the template resolves the camera's server through `DeviceManager`) should be guarded: an exception in your listener loses the alarm and the notification, while an alarm without a server is still an alarm.

12.3 How the sample does it

Trigger (wire name)	Options class	Scope
<code>sample_any_item</code>	<code>AnyItemTriggerSettings</code>	The categories picked in "Where" (<code>requiresSources</code> is true)
<code>sample_contains_word</code>	<code>ContainsWordTriggerSettings</code> (<code>word</code> , <code>@NotBlank</code>)	Every category, including ones created later (<code>requiresSources</code> false, <code>getDynamicSources</code> returns all)

- The alarm always carries the item's category as source; the host shows the category name and attaches the category's camera.
- `getSources()` is cached for 5 s and dropped whenever a category changes (`SampleRuleType.invalidateSources()`, called by `CategoriesController`).
- Trigger wire names carry the plugin prefix: the host translates a trigger as `t(trigger.toUpperCase())` in a namespace shared by every plugin, so the keys are `SAMPLE_ANY_ITEM` and `SAMPLE_CONTAINS_WORD`.
- The host bean-validates the options when a rule is saved: a blank word answers 400 with `{"type":"NOT_BLANK","field":"options.word"}`, shown under the input by the "What" block.
- A category that rules still point at can be deleted once it has no items; those rules keep the source id and stop matching.

12.4 Frontend needs

`admin_events_and_rules_options_block` (the "What" step is empty without it), `admin_events_and_rules_where_block` (your own source picker), `alarms_type_column`, `alarms_message_column`, `alarms_details_message`, `alarms_details_preview` (video), `client_search_alarm_info`, `plugin_notification_item`, and "alarms_available": true in `settings.json`. Chapter 04 lists them.

12.5 Notifications

Notifications normally come from the rule action. `VmsNotificationManager.addNotification(...)` lets a plugin push one directly. A notification type registered through the rule type needs no second registration.

13. Retention

Operators set the retention per category in Admin Center → Settings → Storage Cleaner. The host matches a category by its type id (`logs`, `alerts`, `plugins_data`); the template's `host/CleaningCategories` implements `CleaningCategory` with those ids.

```
context.getCleaner().addDatabaseCleaningTask(CleaningCategories.PLUGINS_DATA, "sample_items",
"created_at");
cleaner.addStorageCleaningTask(CleaningCategories.PLUGINS_DATA, "sample_plugin/images");
cleaner.addCustomTask(() -> ...);
```

The column must hold instants written like the core's (`TIMESTAMP(3)`, `Timestamp.from(Instant)`); index it. The cleaner runs once a day, ten minutes after midnight, and the retention is set in whole days (at least one), so a test of your task takes a night. There is no unregister call: a task lives until VMS restarts and every plugin start adds it again. Deleting rows twice is harmless; keep custom tasks idempotent.

14. Cameras and devices

- `deviceManager.getItem(cameraId)` → `DeviceItemSettings (id, uuid, name, type)`; check `type() == DeviceItemType.CAMERA` when a user hands you a camera id (template: `CategoriesController`).
- `deviceManager.getDeviceById(cameraId).map(Device::getServerId)` → the recording server of the camera, for alarms.
- `deviceManager.isCameraAllowed(user, cameraId)` → respect per-camera permissions when you expose camera-bound data.
- `streamsManager.createSnapshot(uuid)` → a JPEG in storage.

15. Maps and plans

A plugin can add its own placeable objects to maps and floor plans and push live events onto them.

Implement `MapPlanEventEmitter` (`getId()` stable UUID, `getPluginName()`, `getLocaleKey()`, `getIcon()` = URL of an icon in your bundle, `getGroups()`, `getSources()` with parameters, `getEventSettingsClass()`), register it in `init()` and unregister in `terminate()`; on events call

`mapPlanEventManager.processEvent(sourceId, filter, factory)`. The frontend side needs the `map_plan_*` slots of chapter 04. The sample does not use maps; Incoresoft's access-control plugin is the reference and can be shared on request.

16. Logging, threads, configuration

- Logging: SLF4J; the host writes `log/application.log`. Prefix messages with your plugin name.
- Threads: your own daemon executors, named, shut down in `terminate()`.
- Configuration: there is no per-plugin configuration API. Keep settings in your own tables, edited through your Admin Center pages (the sample's categories), or read environment variables. `GlobalSettings` gives read access to the system settings.
- Not available to plugins on 25.1: writing audit records (no custom audit actions), the Event Manager, a background job scheduler (use your own executor).

04 — Frontend

Reference implementation: `templates/frontend-plugin-minimal`. This chapter describes the VMS **25.1** web client.

1. Toolchain and packages

Vue 3.4 · TypeScript · Webpack 5 with `ModuleFederationPlugin` · `vue-router` 4.5 · `vue-i18n` 9.9 · `pinia` 2.1 · SCSS. `npm run typecheck` runs `vue-tsc --noEmit` over the plugin, `.vue` files included. No linter is preconfigured.

`package.json` pins the versions the VMS 25.1 host runs: `vue` 3.4.15, `vue-router` 4.5.1 (4.6 needs Vue 3.5), `vue-i18n` 9.9.1, `pinia` 2.1.7. They are provided by the host at runtime (01 §5), so what compiles also exists at runtime.

The UI packages come from the **public npm registry** and are provided by the host at runtime too:

Package	Use it for
<code>@incoresoft/incoresoft-ui</code>	buttons, tables (<code>ISTable</code>), fields and selects, modals (<code>ISModal</code> , <code>modalService</code>), expand blocks, pagination (<code>ISPagination</code>), loaders
<code>@incoresoft/incoresoft-icons</code>	the icon set (<code>ISIcon*</code>)
<code>@incoresoft/incoresoft-toasts</code>	<code>ToastsService.success</code> / <code>warning</code> / <code>error(localeKey)</code>

Install `^25.1.0` for VMS 25.1 (at the time of writing: `incoresoft-ui` 25.1.489, `incoresoft-icons` 25.1.94, `incoresoft-toasts` 25.1.15). Other `@incoresoft/*` packages you may see in the host are part of the VMS product and not part of the SDK. Component contracts are in each package's `dist/**/*.d.ts`; for example `ISTable` takes `columns: (string | { text, width })[]` and `data: { id, <column>: { value } }[]` and offers the slots `filter`, `actions`, `<column>` and `<column>_header`.

2. `webpack.config.js`

```
new ModuleFederationPlugin({
  name: PLUGIN_NAME,           // 'sample_plugin' == backend PLUGIN_NAME, a valid JS identifier
  filename: 'app.js',          // the host loads <publicPath>app.js
  remotes: { host: 'host@/app.js', styleguide: 'styleguide@/app.js', core: 'core@/app.js' },
  exposes: { './ClientLayoutSamplePlugin': './src/layouts/ClientLayoutSamplePlugin.vue', './clientRoutes': './src/router/clientRoutes.ts', ... },
  shared: getSharedPackages(), // vue, vue-router, vue-i18n, pinia, @incoresoft/*: { singleton: true, import: false }
})
output.publicPath = '/resources/sample_plugin/' // production
```

- `exposes` keys are the `component` values of `component.json` (without `./`).
- `shared ... import: false`: `Vue` and the `@incoresoft/*` packages are never bundled; bundling your own copy breaks reactivity and injection. Every other dependency you add is bundled into your plugin.
- Build with `npm run build` (production mode). A development build bakes a dev-server address into the chunk URLs and does not load inside an installed VMS.

3. `public/component.json`

```
[
  { "component": "ClientLayoutSamplePlugin", "path": "/", "type": "view", "location": null },
  { "component": "AdminLayoutSamplePlugin", "path": "/", "type": "view", "location": "admin" },
  { "component": "VClientHeaderPlusItem", "path": "", "type": "client_header_plus_item", "location": null },
  { "component": "VPluginIcon", "path": "", "type": "plugin_icon", "location": null }
]
```

Field	Meaning
<code>component</code>	Exposed module name. Its <code>default</code> export is used: a Vue component, or for the two util slots a function or object.
<code>type</code>	Slot type (§10). <code>view</code> = routed page.
<code>path</code>	For <code>view</code> : route path relative to <code>/<name></code> (client) or <code>/admin/<name></code> (admin). <code>"/</code> marks the layout; its child routes come from the exposed <code>./clientRoutes</code> / <code>./adminRoutes</code> (export <code>const clientRoutes: RouteRecordRaw[]</code>). Empty for other slots.
<code>location</code>	<code>"admin"</code> → Admin Center route tree (<code>/admin/<name>/...</code>); anything else, <code>null</code> included → client. Ignored by non-view entries.

The backend reads the file from the jar and returns the entries in `GET /api/v1/plugins/extensions`. A change takes effect after you rebuild the jar, update the plugin and reload the page.

4. public/settings.json

```
{
  "title": "Sample Plugin",
  "dev_path": "http://localhost:8030/resources/sample_plugin/app.js",
  "alarms_available": true,
  "routes_permissions": { "/sample_plugin": "ViewSampleItems" },
  "client_permissions": { "search": ["ViewSampleItems"], "plugin_page": ["ViewSampleItems"] }
}
```

Key	Effect
title	The plugin's name as the host shows it: the Roles tab, the layout-cell header, the Search block, the alarm type filter. The client tab title is separate: your header "+" item passes it as a locale key.
dev_path	Used only by Incoresoft's internal development build of the web client. On an installed VMS the bundle is always loaded from the jar. Keep the key; it is harmless.
alarms_available	Adds your plugin to the alarm type filter of the Alarms page and of the notifications bar.
routes_permissions	route prefix → permission id . Guards the plugin's client routes: a route is allowed when it starts with a key whose permission the user holds; otherwise the host redirects to the first allowed key, or to /403 . Key the plugin root (/sample_plugin) so that pages you add later are covered. Admin Center routes are not guarded by it.
client_permissions.plugin_page	Any-of list gating the header "+" item. Empty or missing = everyone.
client_permissions.search	All-of list gating your block on the Search page. [] or missing = everyone.

Write both `client_permissions` lists as JSON arrays. Permission ids must match the backend's `Permission.getId()` .

5. Project layout and conventions

```
src/
nameset.ts          PLUGIN_ID (= module name), PLUGIN_TITLE_KEY, TESTING_ID, PERMISSIONS, modal names prefixed with PLUGIN_ID
layouts/            ClientLayoutSamplePlugin.vue (client tab), AdminLayoutSamplePlugin.vue (Admin Center section with header tabs)
router/            clientRoutes.ts, adminRoutes.ts (+ adminNavigation): RouteRecordRaw[] with relative paths
views/admin/AdminCategoriesView.vue  categories table: search, create / edit (name + camera), delete
views/client/ClientItemsView.vue     items table: pages, category filter, create, delete, video, live updates
components/admin/    VCategoryModal, VAdminHeaderNav, the two rule-wizard blocks
components/alarms/   type column, message column, details message, details preview (video)
components/search/   VSearchFilterBlock, VSearchCard, VSearchInfo, VSearchAlarmInfo
components/notifications/ VNotificationItem
components/cell/     VClientLayoutCell
components/modals/   VItemModal, VItemVideoModal, VSelectCategoryModal
components/          VClientHeaderPlusItem, VAdminSidebarMenuItem, VPluginIcon
composables/useItemsLive.ts the live WebSocket feed, shared by pages and cells
layout-cell/registration.ts side-effect module for the `client_cell_registration` slot
api/                makeRequest.ts (wrapper of core/api), categories.ts, items.ts
locales/            index.ts registers messages/{en,uk,es}.json with the host
models/             alarms.ts, rules.ts, navigation.ts, search.ts – the shapes the host passes in
types/vms-host.d.ts types of the host modules the plugin imports ($7)
```

Rules the template follows:

- **Keep-alive name.** The client layout's component `name` is `'ClientLayout' + PascalCase(module name)` (`sample_plugin` → `ClientLayoutSamplePlugin`); the host caches the tab by that name, and without it your pages remount on every tab switch.
- **Inner keep-alive with an own-route guard.** Both layouts render their pages through `<keep-alive :max="1">` and `<component :is="isOwnRoute ? Component : null" />`. While another tab is active the host keeps your layout alive and its `<router-view>` resolves the other tab's page; the guard keeps your own page cached instead of mounting a foreign one. Use the ternary on `:is`, not `v-if`. Cached pages load their data in `onActivated`, which also runs on the first mount.
- Route names, Pinia store ids and modal names are global in the host: suffix or prefix them with the plugin name. The template has no Pinia store; if you add one, `defineStore( ${PLUGIN_ID}-... )`.
- Add `data-el-name` attributes (`${TESTING_ID}-...`) for UI tests.

6. Talking to the backend

```
import coreApi from 'core/api'
const response = await coreApi.makeRequest({ url: '/api/v2/sample_plugin/items', method: 'GET', params })
```

`makeRequest` prefixes the API base URL, sends the session cookie, toasts `response.data[0].type` for 400 (without a field), 404, 409, 422 and 498, and logs the user out on 401. The template wraps it in `api/makeRequest.ts`, which unwraps `.data`.

Two things to know about `params`:

- arrays are serialised as `ids=[1,2]` (the backend template parses this form);
- **every key is written**, `undefined` and `null` included (`category_id=undefined`). Build the object with the parameters that have a value only, as `api/items.ts` does.

WebSockets: `${coreApi.config.WEBSOCKET_API}/api/v2/ws/sample_plugin/...` through `styleguide/services/WebSocketService`, which reconnects and parses JSON. The template's `composables/useItemsLive.ts` opens one socket for all subscribers, closes it with the last one, and has every subscriber reload after a reconnect, because changes made while the socket was down were never delivered.

The host's confirmation dialog is a modal named `confirmModal`, always mounted: `modalService.show('confirmModal', { title, message, confirm })`. It calls `confirm` and closes without waiting for a returned promise; handle a failed request inside `confirm`.

7. Host modules you can import

The host exposes its modules to plugins through Module Federation; they exist only at runtime, and TypeScript learns about them from `src/types/vms-host.d.ts`, which the template ships. The typed, supported surface for 25.1:

Module	What it gives
<code>core/api</code>	<code>coreApi.config</code> (API, WEBSOCKET_API, STORAGE_URL), <code>coreApi.makeRequest(config)</code>
<code>core/stores/useUserStore</code>	the signed-in user, <code>locale</code> , date and time formats
<code>core/stores/usePermissionStore</code>	<code>hasPermission</code> , <code>hasPluginPermission</code> , per-camera and per-layout checks, <code>isAdmin</code>
<code>core/stores/useDevicesStore</code>	<code>camerasList</code> (cameras the user may see), <code>getCameras()</code>
<code>core/components/AppPlayer</code>	the host player: live, or archive with <code>isPlayback</code> , <code>camera</code> , <code>currentTime</code> , <code>event {start, end}</code>
<code>core/components/VCameraThumbnail</code>	a frame from the archive: <code>cameraId</code> , <code>timestamp</code>
<code>core/formatters/time</code>	<code>formatAsUserDate</code> , <code>formatAsUserTime</code> in the user's format and time zone

Module	What it gives
core/features/layoutCellsRegistry	registerSidebarItem , registerPluginCellEditor (§10 B)
host/stores/useSystemStore	setExtensionsLocales , the list of loaded extensions
styleguide/services/modal-service	modalService.show / hide(name, params)
styleguide/services/WebSocketService	reconnecting WebSocket client
styleguide/services/notification-service	in-app notifications
styleguide/stores/useValidationErrorStore	maps a 400 body onto form fields (getErrorForField , setErrors)

Every other host module resolves as `any` . Prefer the modules above; they are what the sample uses and what Incoresoft keeps stable within a release line. Replace the declaration file when you move to a new release line.

8. Permissions on the client

```
usePermissionStore().hasPluginPermission('ManageSampleItems') // true for root or any role holding it
```

Plus the declarative gates in `settings.json` (§4). The permissions themselves are created by the backend; the frontend only references ids and translates the `PERMISSION_*` keys. Where the host does not filter for you, check yourself: the layout-sidebar entry of `client_cell_registration` is shown to every user, so the template checks `ViewSampleItems` in `acquireContent` and again inside the cell (a layout can be shared with users who lack it). UI checks are a convenience; the backend enforces every permission on its routes.

9. Locales

`useSystemStore().setExtensionsLocales({ en, uk, es })` with flat `KEY: text` maps; the allowed locales are `en` , `uk` and `es` . The host applies the registered maps **once, after all plugins are loaded**, so register them from a module the host loads at start (the template imports `@/locales` in both layouts). Host keys win on conflict, so **prefix your keys** (`SAMPLE_...`). Include: UI strings, `PERMISSION_*` for your permissions, the error `type` keys your API returns, the rule trigger names (`SAMPLE_ANY_ITEM` , `SAMPLE_CONTAINS_WORD` : the host translates a trigger as `t(trigger.toUpperCase())`), and the title of your client tab (`SAMPLE_PLUGIN_TITLE`). Host keys such as `CAMERA` , `VIDEO` , `DELETE` , `SAVE` , `ALL` , `NONE` , `FIELD_REQUIRED` can be used as they are.

Styling: use the UI kit and the host's CSS variables (`--text-color-0...60` , `--background-color-...`); scoped SCSS; no global resets. The plugin icon is an SVG component; some host places recolour it (the layout-cell name pill forces its `fill`).

10. Slots

A slot is a `type` value in `component.json`. The host stores the exposed module's default export in a map keyed by **module name** and renders `map[<name>]` in the matching place. For alarm and notification slots the key is the alarm `type` (`= RuleType.getTypeId()`, by convention the module name). The template implements every slot below except the maps and the roles tab, so the exact props and events can be read from working code.

A. Pages and navigation

Slot type	Where	Contract
<code>view</code>	Vue Router	<code>location: "admin" → /admin/<name>/<path></code> ; otherwise <code>/<name>/<path></code> . The layout entry (<code>path: "/"</code>) takes its children from the exposed <code>./clientRoutes</code> / <code>./adminRoutes</code> .
<code>client_header_plus_item</code>	the header "+" popover	No props. Emit <code>tabClicked</code> with <code>{ id: <name>, type: 'view', title: <locale key>, path: '/<name>/...', icon: <name> }</code> (template: <code>models/navigation.ts</code>). Shown only to users allowed by <code>client_permissions.plugin_page</code> .
<code>plugin_icon</code>	tabs, the layout-cell pill, the Search block list, admin tables, the rule "Where" step	No props. An SVG with a tight <code>viewBox</code> .
<code>admin_sidebar_menu_item</code>	Admin Center sidebar, PLUGINS group	No props. Render a <code><router-link :to="/admin/<name>/..." class="app-sidebar__menu--item"></code> as the template does. Shown to every Admin Center user.

B. Layout cells

Slot type	Where	Contract
<code>client_layout_cell</code>	a layout cell of type <code>plugin</code>	Prop <code>payload: string</code> , the part of the cell content after <code><name>:</code> . The host draws the frame: name pill with <code>settings.title</code> and your icon, controls, padding. Fill the content area only.
<code>client_cell_registration</code>	executed on load	A side-effect module: <code>registerSidebarItem({ id, cellType: 'plugin', group: 'plugins', icon, labelKey, acquireContent })</code> from <code>core/features/layoutCellsRegistry</code> . <code>acquireContent</code> returns the cell content <code>`\${name}:\${payload}`</code> or <code>null</code> to cancel (the template opens its <code>plugin_select_modal</code> and resolves with the chosen category). <code>registerPluginCellEditor(name, editor)</code> adds an edit button to placed cells. The sidebar entry is shown to every user: check the permission in <code>acquireContent</code> .

C. Search page

The host owns the page: period, camera tree, sort order, result list, details panel. Your block does the searching and hands the rows to the host.

`models/search.ts` in the template types the whole contract.

Slot type	Where	Contract
<code>client_search_filter_block</code>	the filter sidebar; shown to users allowed by <code>client_permissions.search</code>	Props: <code>blockKey</code> (= your name), <code>blockName</code> (= <code>settings.title</code>), <code>activeExpanded</code> (v-model), <code>startTime</code> , <code>endTime</code> , <code>cameras</code> (ticked in the host tree), <code>searchOrder</code> ('asc' 'desc'), <code>savedBlockFilters</code> (what you last sent in <code>onChangeFilters</code> , or null), <code>reachEndTrigger</code> (flips: load the next page), <code>refreshTrigger</code> (grows: start over). Emits: <code>update:activeExpanded</code> , <code>onCloseBlock(blockKey)</code> , <code>onChangeFilters(blockKey, filters)</code> (the host clears the list — send it before every new first page), <code>onSetSearchData(blockKey, rows, total)</code> (the host appends the rows), <code>onSetShowLoader(bool)</code> .
<code>client_search_card</code>	over the result card	Prop <code>item</code> : your row plus <code>key</code> and <code>searchedElementType</code> . The host draws the card with a frame from the camera archive when the row has <code>camera_id</code> , <code>timestamp</code> and <code>__SHOW_CAMERA_THUMBNAIL: true</code> .
<code>client_search_info</code>	the details panel	Props <code>item</code> , <code>setSearchPlayerSettings({ camera, startTime, endTime } null)</code> , <code>setSearchDetailsHeaderInfo(title, text)</code> .
<code>client_search_alarm_info</code>	details of one of your alarms in Search → Alarms	Same props; <code>item</code> is the alarm. Needs <code>alarms_available</code> .

D. Alarms and notifications (keyed by alarm type)

Slot type	Where	Contract
<code>alarms_type_column</code>	alarms table "Type" cell; alarm modal	prop <code>alarm</code>
<code>alarms_message_column</code>	alarms table "Message" cell	prop <code>alarm</code>
<code>alarms_details_message</code>	"Message" block of the alarm and notification modals	prop <code>event</code> (the alarm or notification); emit <code>closeModalEvent</code> before navigating away
<code>alarms_details_preview</code>	video block of the alarm and notification modals	prop <code>event</code> . The host adds <code>event.camera_id</code> from the camera of the alarm's source (03 §12.1); the template renders <code>VCameraThumbnail</code> and <code>AppPlayer</code> around <code>event.timestamp</code>
<code>plugin_notification_item</code>	body of a notification card in the notifications bar	prop <code>item</code> ; emits <code>removeNotification(id)</code> and <code>setSystemMessage({ is_desktop, type, title, body, image })</code> , which asks the host for the desktop notification. The host draws the card and its close button; fill the body only

The alarm as your components see it: `id`, `type`, `source_id`, `source_name`, `camera_id`, `server_name`, `rule_id`, `rule_name`, `options` (the rule options), `message` (your rule message), `state`, `priority_level`, `timestamp` (ms), `owner_id`.

E. Admin Center — rule wizard

Slot type	Where	Contract
<code>admin_events_and_rules_options_block</code>	the "What" step, for rules of your type	<code>modelValue</code> (v-model; becomes <code>rule.options</code>), <code>viewMode</code> (<code>'create'</code> <code>'edit'</code> <code>'details'</code> , details is read-only). Emits <code>update:modelValue</code> , <code>showWhereStep</code> / <code>hideWhereStep</code> (the host also listens for the what, when, for and action steps), <code>resetSources</code> (clears the "Where" selection when the trigger changes). The template shows <code>validationStore.getErrorForField('options.word')</code> under its input.
<code>admin_events_and_rules_where_block</code>	the "Where" step, replacing the camera tree	<code>modelValue</code> (v-model, the source UUIDs), <code>options</code> (current rule options), <code>viewMode</code> . The template lists the categories by <code>source_id</code> .
<code>admin_roles_permissions_tab</code>	Roles → role details, an extra tab named <code>settings.title</code>	<code>prop role</code> . Declare the static component option <code>vmsRole: true</code> to receive the VMS role; the host does not listen to events of this tab, so save through your own API. Not in the sample; Incoresoft's access-control plugin uses it for per-door permissions.

F. Modals mounted globally

Slot type	Where	Contract
<code>plugin_select_modal</code>	mounted once by the host, always present	No props. Put <code>ISModal</code> s here and open them with <code>modalService.show('<name>_...', payload)</code> from anywhere; the payload arrives in the modal's <code>beforeOpen</code> event, not as props. The template's category picker for the layout cell lives here.

G. Maps and plans

Not in the sample: the contract below comes from the host code and from Incoresoft's access-control plugin, which can be shared on request.

Slot type	Where	Contract
<code>map_plan_config_block</code>	editor side panel, a tab per plugin of the selected node	<code>config</code> (v-model:config); optionally expose <code>initDefaultConfig()</code> , which the host calls when the node has no settings for your plugin yet. Saved as the node's settings for your plugin and handed to your backend <code>MapPlanEventFilter</code> / <code>MapPlanEventFactory</code> .
<code>map_plan_notification</code>	live overlay on a node	props <code>notification</code> (your <code>MapPlanEvent</code> JSON), <code>config</code> (the node's settings for your plugin)
<code>map_plan_notification_default_config</code>	applied when a node is placed	default export: a function returning the default config object
<code>map_plan_object</code>	your own placeable objects (backend <code>MapPlanEventEmitter</code>)	default export: <code>{ icon: Component, hasFieldOfView: boolean, contextMenu?, nodeOverlay? }</code>

H. Availability

A plugin's components are rendered only while the extension is available: an extension that requires Incoresoft's analytics integration is hidden while that integration is off. The header "+" item additionally needs `client_permissions.plugin_page`, the Search block `client_permissions.search`. The admin sidebar item and the layout-sidebar entry are shown to everyone.

05 — Packaging, versioning, release

1. The artifact

One fat jar (`maven-assembly-plugin`, `jar-with-dependencies`, `appendAssemblyId=false`) named `<artifactId>--<version>.jar` in `build/`, with the manifest attributes of 03 §1.2 and:

```
public/app.js, public/*.js, public/component.json, public/settings.json    ← frontend
META-INF/extensions.idx                                                  ← PF4J
sql/db-changelog.xml, icon.svg                                           ← resources
```

2. Getting the frontend into the jar

1. `npm run build` → `dist/` (`app.js`, the chunks and the copied `public/*.json`).
2. Replace the content of the backend's `src/main/resources/public/` with `dist/`. Replace, do not add: chunk names carry a content hash and old chunks would pile up.
3. `mvn clean package`.

If frontend and backend live in separate repositories, publish `dist/` as a jar whose root contains `public/...` and let the backend depend on it with compile scope, so the assembly unpacks it into the fat jar; the template's `pom.xml` has such a dependency commented out. Both ways give the same jar.

3. Versioning

- **Plugin-Version** is strict SemVer and its **major.minor equals the VMS release line**: `25.1.N` for VMS `25.1`, checked on upload (`ERROR_PLUGIN_NOT_COMPATIBLE` otherwise). Your own product version can go into `Plugin-Name` or into your release notes.
- A plugin is built for one release line. Another line needs its own build: its own library versions, its own `Plugin-Version` series and possibly source changes (26.1 moves to Javalin 7). Keep one branch per release line and never mix libraries of two lines.
- `Plugin-Requires` is not needed and the template omits it (03 §1.2).

Release line	middleware	device-driver-sdk	Javalin	jOOQ	Guice	Jackson	Java	npm @incoresoft/*
25.1	25.1.26	25.1.27	5.6.3	3.18.7	7.0.0	2.14.2	21	^25.1.0

Newer 25.1.x libraries are compatible within the line.

4. CI outline

```
build:      mvn -B clean compile                # backend
            npm ci && npm run typecheck && npm run build    # frontend
test:       mvn test
package:    replace src/main/resources/public with dist ; mvn -DskipTests package
            unzip -p build/*.jar META-INF/extensions.idx    # must list your VmsExtension
deploy:     PATCH /api/v1/plugins/vezha/<Plugin-Id>/update_from_file on your test VMS (§6)
```

5. Host HTTP API for plugin management

Base URL: `https://<host>:2443` (or `http://...:2002`). All plugin-management calls need the **root user**: a session or an API token of root (Admin Center → Settings → API Token), sent as `Authorization: Bearer <token>`.

Errors are the array `[{"type": "<LOCALE_KEY>", "field": <name|null>, "args": [...]}]` (03 §8).

Discovery

`GET /api/v1/plugins/extensions` — anonymous. The entry of the sample plugin:

```
{
  "module_name": "sample_plugin",
  "plugin_id": "vms-sample-plugin",
  "type": "service_plugin",
  "path": "resources/sample_plugin/app.js",
  "title": "Sample Plugin",
  "alarms_available": true,
  "live_view_disabled": false,
  "integration_required": false,
  "components": [ { "component": "ClientLayoutSamplePlugin", "path": "/", "type": "view",
"location": null, "children": null }, ... ],
  "routes_permissions": { "/sample_plugin": "ViewSampleItems" },
  "client_permissions": { "search": "[\"ViewSampleItems\"]", "plugin_page": "
[\"ViewSampleItems\"]" }
}
```

`title`, `alarms_available`, `routes_permissions` and `client_permissions` come from `settings.json` (the `client_permissions` values are passed through as JSON strings), `components` from `component.json`.

Management

Method and path	Purpose
<code>GET /api/v1/plugins/installed</code>	Installed plugins and drivers: <code>id</code> , <code>name</code> , <code>description</code> , <code>version</code> , <code>enabled</code> , <code>type</code> , <code>install_path</code>
<code>POST /api/v1/plugins/vezha</code> (multipart file)	Upload and start a new plugin; 204
<code>PATCH /api/v1/plugins/vezha/{id}/update_from_file</code> (multipart file)	Replace an installed plugin with the uploaded jar; the plugin's tables, settings and alarm rules stay; 204. Any version compatible with the core is accepted, the installed one included
<code>PATCH /api/v1/plugins/vezha/{id}/start · .../stop</code>	Start or stop; the state survives a VMS restart
<code>DELETE /api/v1/plugins/vezha/{id}</code>	Stop, unload, delete the jar. Refused while alarm rules of the plugin's rule type exist
<code>PATCH /api/v1/plugins/vezha/{id}/update</code>	Update to a marketplace version
<code>PUT /api/v1/plugins/vezha/start · .../stop ·</code> <code>PUT /api/v1/plugins/update</code>	All plugins
<code>SSE GET /api/v1/sse/plugins</code>	Plugin started / stopped / installed / uninstalled events; the web client shows its reload prompt on them

`{id}` is the **Plugin-Id**. The `vezha` path segment is historical and means "VMS plugin"; the same routes exist under `driver` for device drivers.

Answers of the upload and the update (400 unless noted):

type	Cause
ERROR_PLUGIN_NOT_COMPATIBLE	Plugin-Version major.minor differs from the core's
ERROR_PLUGIN_IS_INVALID	Plugin-Id missing, Plugin-Version not SemVer, or (update) the jar belongs to another plugin
ERROR_FILE_MUST_HAVE_JAR_EXTENSION	the file name does not end in .jar
ERROR_PLUGIN_IS_ALREADY_INSTALLED	upload of a Plugin-Id that is installed; use the update
ERROR_PLUGIN_NOT_FOUND	update of a Plugin-Id that is not installed
ERROR_PLUGIN_IS_BEING_INSTALLED	another install or update of the same plugin is running
ERROR_PLUGIN_LOADING_FAILED (500)	the jar could not be loaded or started (init() threw, ...). An upload deletes the jar again; an update puts the previous jar back
ERROR_FORBIDDEN (403)	the user is not root

Resources

Path	Content
GET /resources/<module_name>/*	the frontend bundle, served by the plugin (03 §10)
GET /resources/<module_name>/icon	the plugin icon, served by the plugin
GET /resources/<Plugin-Id>/icon	the icon from the manifest, served by the host
GET /storage/<path>	files saved through Storage